

Homework 1 Programming: MDPs and LQRs

CS/Stat 184(0): Introduction to Reinforcement Learning

(Due September 29 at 11:59 pm ET)

Programming Problems

Setup

This assignment uses Python 3 and numpy. For various visualizaitons, you will also need pillow, matplotlib, and pyglet. We recommend installing [Anaconda](#) (a Python package management tool) and then installing these packages in a conda environment using the commands below. Also note that if you are on Linux, you will need to install a gui-backend such as PyQt5 in order for the visualization to work.

To create and then activate the conda environment:

- `conda create -n cs-stat-184 -c conda-forge gym=0.21.0 pyglet=1.5.27 matplotlib pillow`
- `conda activate cs-stat-184`

The pyglet package there also installs and sets up ffmpeg, which is why we prefer Anaconda for use as a package manager (Anaconda also automatically installs numpy and a Python version known to work in this assignment).

If not using Anaconda, please use pip to install gym version 0.21.0, numpy, pillow, matplotlib and pyglet. Please also install ffmpeg. For Mac and Linux (Ubuntu distro) users, using brew install ffmpeg or sudo apt-get install ffmpeg, respectively, should be sufficient. For Windows users, please follow the instructions [here](#). The download link on that page is broken so please download ffmpeg-git-full.7z found [here](#).

4 Programming Problem for Tabular MDPs (20 pts)

4.1 Package Requirements

Please use the conda environment or alternative setup you created above.

4.2 Deliverables

For this assignment, we will ask you to work in a 4x4 gridworld. This project consists of the following 4 files:

- `MDP.py`: This contains the MDP class which is used to represent our gridworld MDP. It contains the states, actions, rewards, transition probabilities, and the discount factor.
- `visualize.py`: This file can be used to help visualize the V-function and policy. There are also functions that can be used for stepping through value and policy iteration.
- `test.py`: Test case files to check some of your work (see 5.6). Run this with the command `python test.py`.
- `DP.py`: All student code goes here. This file will be used for implementing value iteration, exact policy iteration, and approximate policy iteration.

You can find the four files on Canvas. Your job is to implement all the functions with a TODO comment in `DP.py`. More detail is given below. For submission, please do the following:

- Run the visualization for the value iteration up to iteration 20. Save the figure showing the Q-values as an image, and name it `Q20`. Additionally, save the figure showing the max policy and its corresponding value function as an image, and name it `maxPolicy20`.
- Run the visualization for exact and iterative policy iteration for 5 iterations. Save both figures as images with names `exact5` and `iterative5`, respectively.
- Zip the above 4 figures along with the 4 python files listed above into one folder, name it `submit.zip`, and submit on Gradescope.

4.3 Understanding the MDP

4.3.1 States and rewards

There are 17 states, labeled 0 through 16, as shown below (each box corresponds to a state).

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

16

State 15 (the green box) shown above is the “goal” state, and has a reward of 100. All the white boxes have reward -1, while the orange boxes are “bad” states that have -70 reward. While our gridworld is 4×4 , we need an extra state (state 16) to be the terminal state. This state has reward 0 and is absorbing, which means once we enter the terminal state, we cannot leave, i.e. any action takes us back to the terminal state. Additionally, once we hit the goal state, we automatically transition to the terminal state. This terminal state allows us to model tasks which terminate at a finite time using the infinite-horizon MDP formulation.

Note that the rewards here are outside the range $[0,1]$. Although many theoretical results assume the reward lies within $[0,1]$, in practice, the reward can be any scalar value.

Since we are using a simple state representation, we only need to store the number of states in the MDP class. The rewards, R , is a $|A| \times |S|$ numpy array (note the first index is the action unlike in class). That is, $r(s, a) = R[a, s]$. The rewards in this MDP are deterministic.

4.3.2 Actions

There are 4 actions that can be taken in each state. These actions are up (0), down (1), left (2), right (3).

4.3.3 Transition probabilities

The transition probabilities are easy to define for a gridworld MDP, which makes it a good setting for implementing value iteration, policy evaluation, and policy iteration. The transition matrix P , in `MDP.py`, contains the transition probabilities. Note that the transitions

are stochastic (i.e. moving right from state 0 isn't guaranteed to move you to state 1, you could move to state 4 or stay in the same spot). P is an $|A| \times |S| \times |S|$ NumPy array, and the transition probability $P(s'|s, a)$ can be found by indexing: $P(s'|s, a) = P[a, s, s']$.

4.3.4 Policies

As in class, denote a policy as π (or pi in code). We will only consider deterministic policies in this question, and so will represent a policy using a NumPy array of size $|S|$. The s^{th} entry in such a vector represents the (deterministic) action taken by the policy at state s .

4.4 Value Iteration with the Q-function

4.4.1 TODOs

For value iteration, there are 6 functions (TODOs) to complete.

- computeVfromQ
- computeQfromV
- extractMaxPifromQ
- extractMaxPifromV
- valueIterationStep
- valueIteration

4.4.2 Switching between Q and V functions [3 points]

Given V^π and Q^π , please implement computeVfromQ and computeQfromV using the equations from class

$$\begin{aligned}Q^\pi(s, a) &= r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)} V^\pi(s'), \\V^\pi(s) &= Q^\pi(s, \pi(s)).\end{aligned}$$

Notice that similar to our rewards R , we represent the Q-function, Q^π , as an $|S| \times |A|$ NumPy array Q , where $Q^\pi(s, a) = Q[s, a]$. (Given this, what should the value function look like?)

4.4.3 Computing max policy [3 points]

From class, we know that if we have the optimal Q-function, Q^* , we can find the optimal policy

$$\pi^*(s) = \operatorname{argmax}_a Q^*(s)$$

We can also use this operation at the end of value iteration on Q^t (our final Q values) and also during the policy improvement stage of policy iteration. Please implement extractMaxPifromQ and extractMaxPifromV.

4.4.4 Value Iteration [4 points]

First, implement valueIterationStep. This implements the following equation

$$\forall s, a : Q^{t+1}(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)} \max_{a'} Q^t(s', a')$$

Q^t is passed in to this function while the rewards and transition probabilities can be accessed from the attributes. While not strictly necessary, try implementing this function without for-loops.

Next, implement valueIteration following the specification. This should consist of running valueIterationStep in a loop until the threshold is met. Then, extract the policy from the final Q-function and compute the corresponding V-function. Return the policy, V-function, iterations required until convergence, and the final epsilon.

4.5 Policy Evaluation

In this section, we ask you to implement both exact policy evaluation and iterative policy evaluation.

4.5.1 TODOs

For value iteration, there are 2 functions (TODOs) to complete.

- `exactPolicyEvaluation`
- `approxPolicyEvaluation`

4.5.2 Exact Policy Evaluation [3 points]

The Bellman equation for deterministic policies, π , from class is as follows

$$\begin{aligned} V^\pi(s) &= r(s, \pi(s)) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, \pi(s))} V^\pi(s') \\ &= r(s, \pi(s)) + \gamma \sum_{s'} \mathbb{P}(s'|s, \pi(s)) V^\pi(s') \end{aligned}$$

Using vector notation, this can be written as the linear equation

$$V = R^\pi + \gamma P^\pi V$$

Hint: Use `extractRpi` and `extractPpi` to easily extract $r(s, \pi(s))$ and $P(s'|s, \pi(s))$ in vector form.

By solving this, we can compute V^π . Do not use matrix inversion for this. Instead use `np.linalg.solve`.

4.5.3 Iterative Policy Evaluation [3 points]

Implement approximate policy evaluation following the algorithm given in class. Each step of iterative policy evaluation involves computing V^{t+1} given the current estimate V^t

$$V^{t+1} = R^\pi + \gamma P^\pi V^t$$

Run this algorithm until convergence (as defined by the tolerance).

4.6 Policy Iteration

4.6.1 TODOs [4 points]

For policy iteration, there are 2 functions (TODOs) to complete.

- `policyIterationStep`
- `policyIteration`

Finally, we will move onto policy iteration. This is another iterative algorithm in which we cycle between policy evaluation and policy improvement to continuously improve our policy. Note the difference between this algorithm and value iteration. In value iteration, we did not compute a policy until we computed the final estimate of the optimal Q-function.

First, implement the function `policyIterationStep`. Given the current policy π^t , compute the value-function for π^t using policy evaluation. This can be done using either exact or iterative policy evaluation. With V^{π^t} , compute the new policy.

Once `policyIterationStep` is implemented, implement `policyIteration` in the same style as `valueIteration`. That is, continuously call `policyIterationStep` until convergence. Convergence in this case occurs when the policy improvement step does not change our current policy.

4.7 Testing

We have written some basic tests for you to test your implementation. These are separated into tests for VI, PE, and PI. Please note that these tests are basic and do not guarantee that your implementation is correct.

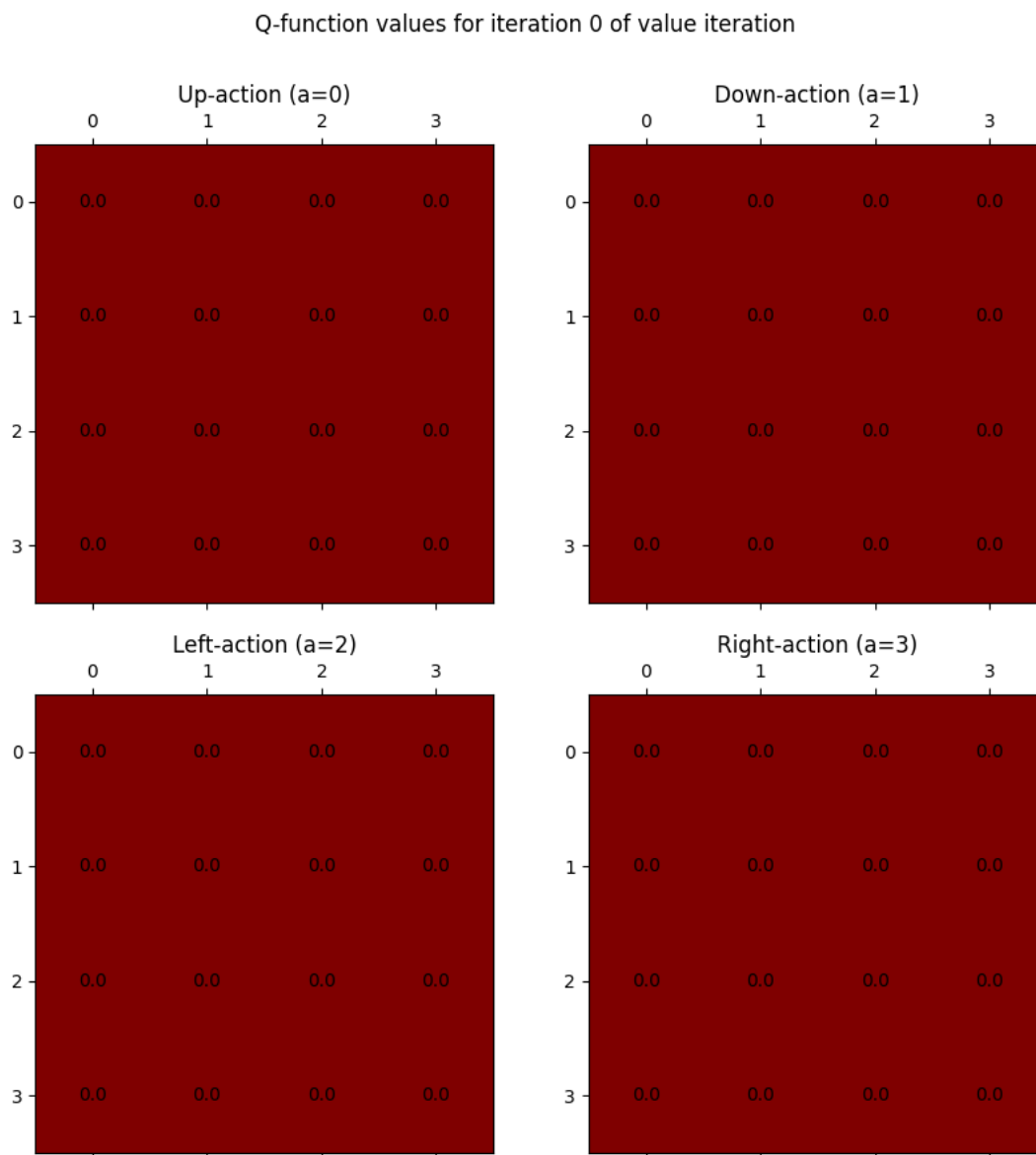
4.8 Visualization

We have also given you code for visualizing value iteration and policy iteration.

4.8.1 Value Iteration

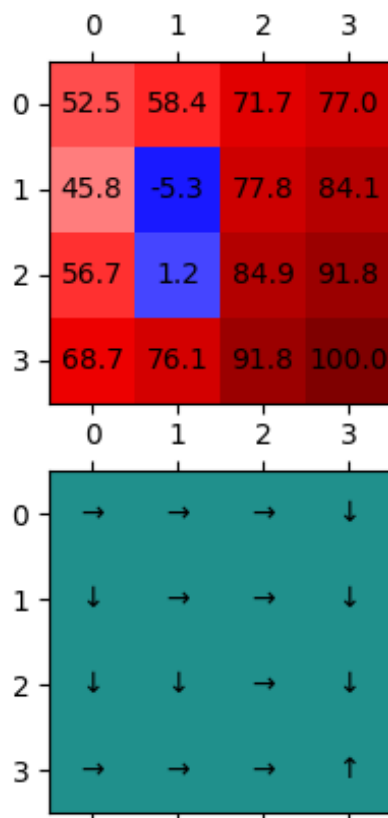
After implementing value iteration, run `python visualize.py VI`. This will allow you to visualize value iteration from an initial Q^0 of all zero's. To change the initial Q^0 , change line 152 in `visualize.py`.

Running this command should open the following window.



This shows the Q-values for each of the 4 actions. To step through an iteration of value iteration, press the right arrow key. The grids should update as you step through value iteration.

Normally, value iteration runs until convergence and then the final policy is extracted. However, we have added extra visualization to allow you to see how the policy changes. For any iteration, t , press the enter key to compute $\pi(s) = \arg \max_a Q^t(s, a)$ and V^π . A new window should show up showing the V-function and policy. See below.



To continue stepping through value iteration, exit this window and continue pressing the right arrow key on the main figure window.

4.8.2 Policy Iteration

To visualize policy iteration, run `python visualize.py PI_exact` or run `python visualize.py PI_approx`. Again, use the right arrow key to step through policy iteration. To change π^0 , change line 155 or 158 of `visualize.py`

5 Local Linearization Approach for Controlling CartPole [50 points]

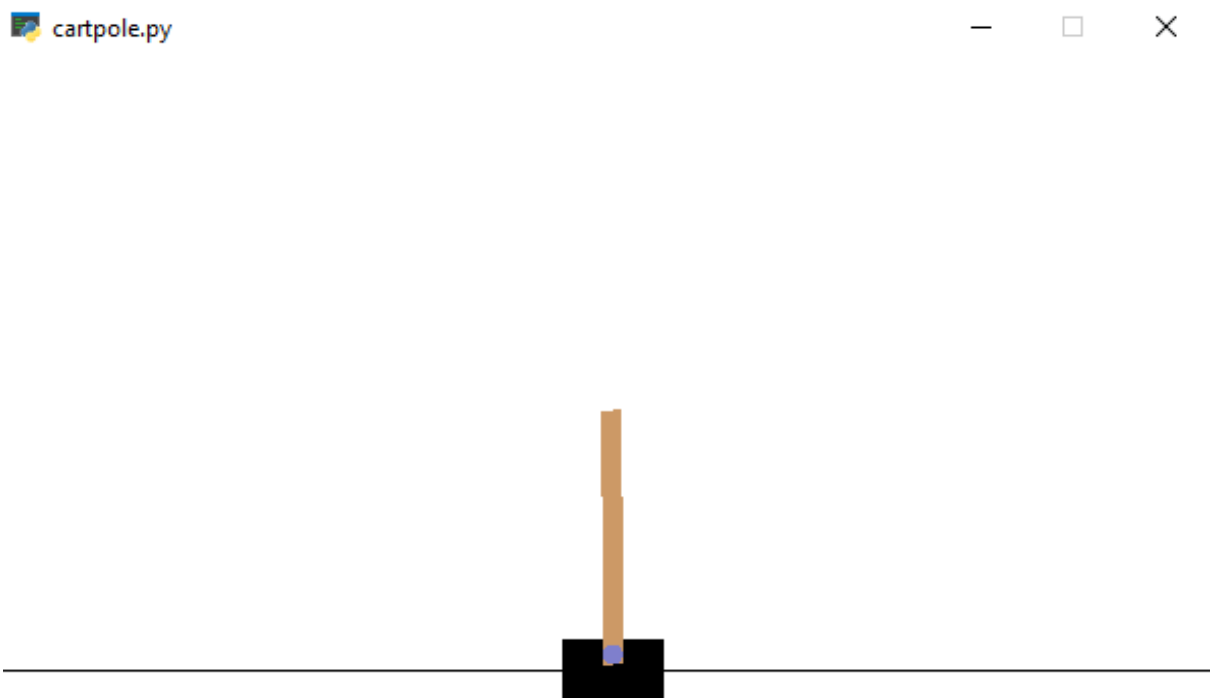
5.1 Package Requirements

Please use the conda environment or alternative setup you created above.

5.2 Introduction

For this assignment, we will ask you to implement LQR in order to solve a cartpole environment. While we are using a custom cartpole environment, it would be useful to go through the following OpenAI Gym introduction. It may also be useful to look up how gym environments work in general. You may find this [tutorial](#) useful.

OpenAI Gym contains many environments that can be used for testing RL algorithms. Additionally, it allows users to create their own custom environments. All these environments share a standard API, making it easy to test different algorithms on different environments. As mentioned earlier, we are using a custom version of the cartpole environment. The goal here is to keep the pole on the cartpole upright by applying a variable force either left or right on the cartpole as shown below.



In the standard cartpole environments, the only possible actions are to apply a force left or right. The amount of force applied is constant. In our environment, we allow variable force to be applied.

There are several files in this project.

- `LQR.py`: This contains the code to find the optimal control policy for an LQR given the respective matrices.
- `finite_difference_method.py`: This file is used to finite difference approximations. These will be used to help compute the reward and transition matrices from the environment which we then pass to the LQR.
- `test.py`: Test case files. Run this with `python test.py`. These will contains tests for different parts of the code.
- `cartpole.py` This contains the standard loop for interacting with the environment.

You can find the files on Canvas

5.3 Deliverables

Your task is to implement local linearization control for the cartpole non-linear setting. You will need to complete the functions in

- `finite_difference_method.py`
- `cartpole_controller.py`
- `LQR.py`

Once these are done, you will need to run some code for testing and visualizing the cartpole. You will turn in the test output and video. More instructions can be found at the bottom of this handout.

5.4 Background on local linearization approach for nonlinear control

Consider the following general nonlinear control problem:

$$\begin{aligned} \min_{\pi_0, \dots, \pi_{H-1}} \quad & \sum_{t=0}^{H-1} c(x_t, u_t) \\ \text{subject to:} \quad & x_{t+1} = f(x_t, u_t), u_t = \pi_t(x_t), x_0 \sim \mu_0; \end{aligned}$$

where $f : \mathbb{R}^{n_x} \times \mathbb{R}^{n_u} \mapsto \mathbb{R}^{n_x}$.

In general, the cost $c(x, u)$ could be anything. Here we focus on a special instance where we try to keep the system stable around some stable point (x^*, u^*) , i.e. our cost function $c(x, u)$ penalizes the deviation to (x^*, u^*) , i.e., $c(x, u) = \rho(x_t - x^*) + \rho(u_t - u^*)$, where ρ could be some distance metric such as ℓ_1 or ℓ_2 distance.

To deal with nonlinear f and non-quadratic c , we use the linearization approach here. Since the goal is to control the robot to stay at the pre-specified stable point (x^*, u^*) , it is reasonable to assume that the system is approximately linear around (x^*, u^*) , and the cost is approximately quadratic around (x^*, u^*) . Namely, we perform first-order Taylor expansion of f around (x^*, u^*) , and we perform second-order Taylor expansion of c around (x^*, u^*) .

The linear approximation of f is:

$$f(x, u) \approx A(x - x^*) + B(u - u^*) + f(x^*, u^*).$$

Here A and B are Jacobians, i.e.,

$$A \in \mathbb{R}^{n_x \times n_x}, \quad A[i, j] = \left. \frac{\partial f[i](x, u)}{\partial x[j]} \right|_{(x, u) = (x^*, u^*)}$$

$$B \in \mathbb{R}^{n_x \times n_u}, \quad B[i, j] = \left. \frac{\partial f[i](x, u)}{\partial u[j]} \right|_{(x, u) = (x^*, u^*)}$$

where $f[i](x, u)$ stands for the i -th entry of $f(x, u)$, and $x[i]$ stands for the i -th entry of the vector s . Similarly, the quadratic approximation of c around (x^*, u^*) is:

$$c(x, u) \approx \frac{1}{2} \begin{bmatrix} x - x^* \\ u - u^* \end{bmatrix}^\top \begin{bmatrix} Q & M \\ M^\top & R \end{bmatrix} \begin{bmatrix} x - x^* \\ u - u^* \end{bmatrix} + \begin{bmatrix} q \\ r \end{bmatrix}^\top \begin{bmatrix} x - x^* \\ u - u^* \end{bmatrix} + c(x^*, u^*)$$

where the gradients/Hessians q, r, Q, R, M are the vectors/matrices containing the first/second derivatives of c respectively, evaluated at (x^*, u^*) :

$$Q \in \mathbb{R}^{n_x \times n_x} : \quad Q[i, j] = \left. \frac{\partial^2 c(x, u)}{\partial x[i] \partial x[j]} \right|_{(x, u) = (x^*, u^*)}, \quad R \in \mathbb{R}^{n_u \times n_u} : \quad R[i, j] = \left. \frac{\partial^2 c(x, u)}{\partial u[i] \partial u[j]} \right|_{(x, u) = (x^*, u^*)}$$

$$M \in \mathbb{R}^{n_x \times n_u} : \quad M[i, j] = \left. \frac{\partial^2 c(x, u)}{\partial x[i] \partial u[j]} \right|_{(x, u) = (x^*, u^*)}, \quad q \in \mathbb{R}^{n_x} : \quad q[i] = \left. \frac{\partial c(x, u)}{\partial x[i]} \right|_{(x, u) = (x^*, u^*)}$$

$$r \in \mathbb{R}^{n_u} : \quad r[i] = \left. \frac{\partial c(x, u)}{\partial u[i]} \right|_{(x, u) = (x^*, u^*)}.$$

We are almost ready to compute a control policy using A, B, Q, R, M, q, r . One potential issue here is that the original cost function $c(x, u)$ may not be even convex. Thus the Hessian matrix $W := \begin{bmatrix} Q & M \\ M^\top & R \end{bmatrix}$ may not be a positive definite matrix. We will apply further approximation to make it a positive definite matrix. Denote the eigen-decomposition of W as $W = \sum_{i=1}^{n_x+n_u} \sigma_i v_i v_i^\top$ where σ_i are eigenvalues and v_i are corresponding eigenvectors. We approximate W as follows:

$$W \approx \sum_{i=1}^{d_x+d_u} \mathbf{1}\{\sigma_i > 0\} \sigma_i v_i v_i^\top + \lambda I, \quad (1)$$

where $\lambda \in \mathbb{R}^+$ is some small positive real number for regularization. That is, we only keep the positive eigenvalues and zero out the rest, and also offset our eigenvalues by λ , which ensures that after approximation, we get a W that is positive definite with minimum eigenvalue lower bounded by λ .

5.4.1 TODOs:

Before moving on, please review the concepts of gradient, Hessians, and positive definite matrices above.

5.5 Setup of Simulated CartPole

The simulated CartPole environment consists of:

- Nonlinear deterministic dynamics $x_{t+1} = f(x_t, u_t)$,
- A potentially non-quadratic cost function $c(x, u)$ that penalizes the deviation of the state from the balance point (x^*, u^*) , where $u^* = 0$ (the system is stable), and x^* represents a state in which the pole is vertical and the cart is in a pre-specified position.

A state x_t is a 4-dimensional vector defined as

$$x_t = \begin{bmatrix} z_t \\ v_t \\ \theta_t \\ \omega_t \end{bmatrix}.$$

It consists of the position of the cart, the speed of the cart, the angle of the pole (in radians), and the angular velocity of the pole. The action u_t is a 1-dimensional vector corresponding to the horizontal force applied on the cart.

Through this section, we assume that we have black-box access to c and f , i.e., we can feed any (x, u) to f and c , we will get two scalar returns which are $f(x, u)$ and $c(x, u)$ respectively. Namely, we do not know the analytical math formulation of f and c (e.g., imagine that we are trying to control some complicated simulated humanoid robot. The simulator is the black-box f).

In this assignment we will use our customized OpenAI gym CartPole environment provided in the following repository. The environment is under `env` directory. The goal is to finish the implementation of `cartpole_controller.py` which contains a class to compute the locally linearized optimal policy of our customized CartPole environment. We also provide other files to help you get started.

5.5.1 TODO:

Please review the files for the programming assignment.

5.6 Using Finite Difference for Taylor Expansion

Since we do not know the analytical form of f and c , we cannot directly compute the analytical formulations for the gradients, Jacobians, and Hessians. However, given black-box access to f and c , we can use finite differences to approximately compute these quantities.

Below we first explain the finite differencing approach for approximately computing derivatives. Your task is to use finite differencing methods to compute A, B, Q, R, M, q, r .

To illustrate finite differencing, assume that we are given a function $g : \mathbb{R} \mapsto \mathbb{R}$. Given any $\alpha \in \mathbb{R}$, to compute $g'(\alpha)$, we can perform the following process:

$$\text{Finite difference for first derivative: } \hat{g}'(\alpha) := \frac{g(\alpha + \delta) - g(\alpha - \delta)}{2\delta},$$

for some $\delta \in \mathbb{R}^+$. Note that by the definition of derivative, we know that when $\delta \rightarrow 0^+$, the approximation approaches $g'(\alpha)$. In practice, δ is a tuning parameter: we do not want to set it to 0^+ due to potential numerical issues. We also do not want to make it too large, as this would give a bad approximation of $g'(\alpha)$.

With $\hat{g}'(\alpha)$ as a black-box function, we can compute the second derivative using finite differencing once more:

$$\text{Finite difference for second derivative: } \hat{g}''(\alpha) := \frac{\hat{g}'(\alpha + \delta) - \hat{g}'(\alpha - \delta)}{2\delta}$$

Note that to implement the above second derivative approximator $\hat{g}''(\alpha)$, we need to first implement the function $\hat{g}'(\alpha)$ and treat it as a black-box function. You can see that we need to query f twice to compute $g'(\alpha)$, and we need to query it four times to compute $g''(\alpha)$.

In the multidimensional case, similar ideas can be used to approximate gradients, Jacobians, and Hessians.

At this end, using the provided Cartpole simulator which has black-box access to f and c , and the goal balance point x^*, u^* , to compute A, B, Q, R, q, r , with the provided value for δ .

5.6.1 TODO: [10 points]

We provide minimum function skeletons and some tests for the finite difference method in the file `finite_difference_method.py`. Complete the implementation of the gradient, Jacobian and Hessian functions. (In the next section, we will use these to compute A, B, Q, R, M, q , and r .)

5.7 Computing locally optimal control

With A, B, Q, R, M, q, r computed from finite differencing, let us first check if $\begin{bmatrix} Q & M \\ M^\top & R \end{bmatrix}$ is a positive definite matrix. If it's not positive definite (PD), your LQR formulation may run into the case where a matrix inverse does not exist, and numerically you will observe NaN (not a number) errors as well. In this case, let's use the trick in Eq. 1. Denote $W := \begin{bmatrix} Q & M \\ M^\top & R \end{bmatrix}$.

We now are ready to solve the following linear quadratic system:

$$\min_{\pi_0, \dots, \pi_{H-1}} \sum_{t=0}^{H-1} \frac{1}{2} \begin{bmatrix} x_t - x^* \\ u_t - u^* \end{bmatrix}^\top W \begin{bmatrix} x_t - x^* \\ u_t - u^* \end{bmatrix} + \begin{bmatrix} q \\ r \end{bmatrix}^\top \begin{bmatrix} x_t - x^* \\ u_t - u^* \end{bmatrix} + c(x^*, u^*), \quad (2)$$

$$\text{subject to } x_{t+1} = Ax_t + Bu_t + m, u_t = \pi_t(x_t), \quad x_0 \sim \mu_0. \quad (3)$$

By rearranging some terms, we can rewrite the above program in the following format.

We can expand the cost function as

$$\begin{aligned} c(x_t, u_t) &\approx \frac{1}{2} \begin{bmatrix} x_t - x^* \\ u_t - u^* \end{bmatrix}^\top W \begin{bmatrix} x_t - x^* \\ u_t - u^* \end{bmatrix} + \begin{bmatrix} q \\ r \end{bmatrix}^\top \begin{bmatrix} x_t - x^* \\ u_t - u^* \end{bmatrix} + c(x^*, u^*) \\ &= x_t^\top \left(\frac{Q}{2} \right) x_t + u_t^\top \left(\frac{R}{2} \right) u_t + x_t^\top M u_t + (q^\top - x^{*\top} Q - u^{*\top} M^\top) x_t + (r^\top - u^{*\top} R - x^{*\top} M) u_t \\ &\quad + \left(c(x^*, u^*) + \frac{1}{2} x^{*\top} Q x^* + \frac{1}{2} u^{*\top} R u^* + x^{*\top} M u^* - q^\top x^* - r^\top u^* \right) \end{aligned}$$

Next, let us expand out the transition function, f . As in section 5.4, we perform a first-order Taylor expansion of f around (x^*, u^*) , so our transition is defined by

$$\begin{aligned} x_{t+1} &= A(x - x^*) + B(u - u^*) + f(x^*, u^*) \\ &= Ax - Ax^* + Bu - Bu^* + f(x^*, u^*) \\ &= Ax + Bu + (f(x^*, u^*) - Ax^* - Bu^*) \end{aligned}$$

Thus, let us define the following variables:

$$\begin{aligned} Q_2 &= \frac{Q}{2} \\ R_2 &= \frac{R}{2} \\ q_2^\top &= q^\top - x^{*\top} Q - u^{*\top} M^\top \\ r_2^\top &= r^\top - u^{*\top} R - x^{*\top} M \\ b &= c(x^*, u^*) + \frac{1}{2} x^{*\top} Q x^* + \frac{1}{2} u^{*\top} R u^* + x^{*\top} M u^* - q^\top x^* - r^\top u^* \\ m &= f(x^*, u^*) - Ax^* - Bu^* \end{aligned}$$

Now we can rewrite our formulation as:

$$\begin{aligned} \min_{\pi_0, \dots, \pi_{H-1}} \sum_{t=0}^{H-1} x_t^\top Q_2 x_t + u_t^\top R_2 u_t + x_t^\top M u_t + q_2^\top x_t + r_2^\top u_t + b \\ \text{subject to } x_{t+1} = Ax_t + Bu_t + m, u_t = \pi_h(x_t), x_0 \sim \mu_0 \end{aligned}$$

This is a slightly more general formulation than the one we saw in class. Here, the cost function contains an additional second order term $x_t^\top M u_t$, first-order terms $q_2^\top x_t + r_2^\top u_t$, and zeroth order term b , and the dynamics contains zeroth order term m .

Let us modify the dynamic programming algorithm we showed in class for computing V_t^* , Q_t^* , and π_t^* for this problem. We first claim that these can be represented as follows:

$$\begin{aligned} Q_t^*(x, u) &= x^\top C_t x + u^\top D_t u + x^\top E_t u + f_t^\top x + g_t^\top u + h_t \\ \pi_t^*(x) &= K_t x + k_t \\ V_t^*(x) &= x^\top P_t x + y_t^\top x + p_t \end{aligned}$$

(You might find it helpful to annotate the shapes of the matrices and vectors.) These parameters can be computed as follows:

- For Q_t^* :

$$\begin{aligned} C_t &= Q_2 + A_t^\top P_{t+1} A_t \\ D_t &= R_2 + B_t^\top P_{t+1} B_t \\ E_t &= M + 2A_t^\top P_{t+1} B_t \\ f_t &= q_2 + A_t^\top y_{t+1} + 2A_t^\top P_{t+1} m \\ g_t &= r_2 + B_t^\top y_{t+1} + 2B_t^\top P_{t+1} m \\ h_t &= b + m^\top P_{t+1} m + m^\top y_{t+1} + p_{t+1} \end{aligned}$$

- For π_t^* :

$$\begin{aligned} K_t &= -\frac{1}{2} D_t^{-1} E_t^\top \\ k_t &= -\frac{1}{2} D_t^{-1} g_t \end{aligned}$$

- For V_t^* :

$$\begin{aligned} P_t &= C_t + K_t^\top D_t K_t + E_t K_t \\ y_t &= 2K_t^\top D_t^\top k_t + E_t k_t + K_t^\top g_t + f_t \\ p_t &= k_t^\top D_t^\top k_t + k_t^\top g_t + h_t \end{aligned}$$

It is also your task to decide the base case of the dynamic programming algorithm.

5.7.1 TODO: [30 points]

Please complete the functions in `lqr.py`. using the formulation derived above. We recommend implementing the functions in order. For the LQR function specifically, you will need to compute the optimal policies for time steps $H - 1, \dots, 0$. The general pipeline at timestep t is to derive Q_t^* , find π_t^* , then finally derive V_t^* before moving on to the next time-step.

Please complete the function `compute_local_policy` in `cartpole_controller.py`. The function computes the linear locally optimal control policy of the CartPole environment. You should use the finite difference function as well as the `lqr` functions to implement this function.

There are a few tests in `test.py` for testing your functions.

Clarification: For the function `compute_q_params` in the provided code file, the parameter Q represents Q_2 in the formula above.

Note: It may be useful to occasionally check that the numpy arrays you work with are of type `np.float64`. While you shouldn't need to explicitly cast things as all our tests initialize arrays to be of type `np.float64`, occasionally checking for it can avoid type issues. The gym environment is built around `np.float64` and using other types can cause incorrect results.

5.8 Test the performance

Given policies $\pi_0, \dots, \pi_{H-1}$ that computed from the previous section, we will evaluate its performance by executing it on the real system f and real cost c . To generate a H -step trajectory, we first sample $x_0 \sim \mu_0$, and then take $u_t = \pi_t(x_t)$, and then call the black-box f to compute $x_{t+1} = f(x_t, u_t)$ till $t = H-1$. This gives us a trajectory $\tau = \{x_0, u_0, x_1, u_1, \dots, x_{H-1}, u_{H-1}\}$. The total cost of the trajectory is $C(\tau) := \sum_{t=0}^{H-1} c(x_t, u_t)$. Since we have randomness in μ_0 , we need to draw N such i.i.d trajectories, $\tau^1 \dots \tau^N$, and compute the average, i.e., $\sum_{i=1}^N C(\tau^i)/N$. As N approaches to $+\infty$, we have (by the law of large numbers):

$$\sum_{i=1}^N C(\tau^i)/N \rightarrow \mathbb{E}_{x_0 \sim \mu_0} \left[\sum_{t=0}^{H-1} c(x_t, u_t) \mid u_t = \pi_t(x_t), x_{t+1} = f(x_t, u_t) \right].$$

In the test file `test.py`, we provide several difference initialization distributions. These distributions are ordered based on the distance between the means to the goal (x^*, u^*) . Intuitively, we should expect that our control perform worse when the initial states are far away from the taylor-expansion point (x^*, u^*) , as our linear and quadratic approximation become less accurate. Testing your computed policies on these difference initializations and report the performances for all initializations by running `test.py`.

This file also contains tests for testing your finite difference methods and the LQR functions.

5.8.1 TODO [7 points]

Run `test.py` and take a screenshot of the output costs printed from `test_policy`. The costs should be as it can vary depending on the version of packages used (even with the environment being seeded). However, your costs should fall in the following ranges

- Case 0 < 10
- Case 1 < 200
- Case 2 < 1000

- Case 3 < 2000
- Case 4 < 5000
- Case 5 = ∞
- Case 6 = ∞
- Case 7 = ∞

Please know that these are loose upper bounds and the actual costs for a correct implementation may be much lower than the upper bound listed (apart from Case 5-7).

Please briefly explain why you think the costs are different for each case.

5.9 Visualization

We can now see the controller in action. Using the visualization tool provided by Open AI Gym, provide a video demonstration of your policy with the given initial distribution by running `cartpole.py`. The generated mp4 video file should be under the directory `./gym-results`. Save this as `cartpole.mp4`.

If the visualization doesn't work (which is required for submission) locally, then it may be useful to port your code over to a colab notebook and visualize things there. The tutorial on OpenAI Gym in the next section has code for visualization.

5.9.1 TODO [3 points]

Please check that the mp4 player can work. There have been known issues in the past with saving the mp4 so if this occurs, please reach out to a TA. Additionally, save the cost of the cartpole and include it in your writeup.

5.10 Comparison

You can compare the LQR policy to a very simple PID policy by changing `flag='LQR'` to `flag='PID'` in `cartpole.py`. You can also investigate how the two different policies perform when you change the initial condition `s_init`. Which works better? There are no deliverables for this section.

5.11 Submission

Please submit a zip file containing your implementation along with your video file name `cartpole.mp4` organized as follows.

```
YOUR_HUID/
├── Answers.pdf
├── README.md
├── __init__.py
└── cartpole.mp4
```

```
|
├─ cartpole.py
├─ cartpole_controller.py
├─ finite_difference_method.py
├─ lqr.py
├─ test.py
├─ env/
│   └─ __init__.py
│   └─ cartpole_control_env.py
```

where Answers.pdf contains the a screenshot of the cost of the cartpole simulations and your explanation from Section 5.8.1 as well as the cost from 5.9.1.